

Introduction to Supervised Learning & Model Selection

Week 1

Welcome to Statistical Machine Learning

What is machine learning?

Machine learning is the science of learning patterns from data.

The goal is to use data to:

- understand relationships between variables
- make predictions for new observations
- discover hidden structures in data

Examples:

- Predict whether a patient will respond to a treatment
- Predict house prices from characteristics
- Identify spam emails
- Group customers based on purchasing behaviour

Types of Machine Learning Problems

Supervised Learning: Machine is given data and examples of what to predict.

Unsupervised Learning: Machine is given data, but not what to predict.

Reinforcement Learning: Machine gets data by interacting with an environment and tries to minimize cost.

The Basics of Supervised Learning

We have an outcome measurement, Y (also called the dependent variable, response, or target).

We have a vector of p predictor measurements, X (also called the inputs, regressors, covariates, features, or independent variables).

We have training data $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$ which are observations (also called instances or realizations) of the measurement (X, Y) .

In **regression** problems, Y is quantitative (e.g. price, blood pressure).

In **classification** problems, Y takes values in a finite, unordered set (survived/died, digit 0-9, cancer class of tissue sample)

Main Tasks in Supervised Learning

On the basis of the training data, we would like to:

- **Prediction:** accurately predict future outcomes (Y).
- **Estimation:** understand how features (X) affect the outcome (Y).
- **Model Selection:** find the best model for predicting the outcome (Y), or which features (X) affect the outcome (Y).
- **Inference:** assess the quality of our prediction, estimation, and model selection.

Examples of Supervised Learning

Spam Emails Detection

Consider a dataset consisting of 4600 emails sent to an individual.

- Each is labeled as **spam** or **email**

The goal is to build a customized spam filter.

We use the relative frequency of 57 words and punctuation marks in the email as **features**.

Type	Tatiana	you	UofT	free	!	edu	remove
Spam	0	2.26	0.02	0.54	0.51	0.01	0.28
Email	1.27	1.27	0.90	0.07	0.11	0.29	0.01

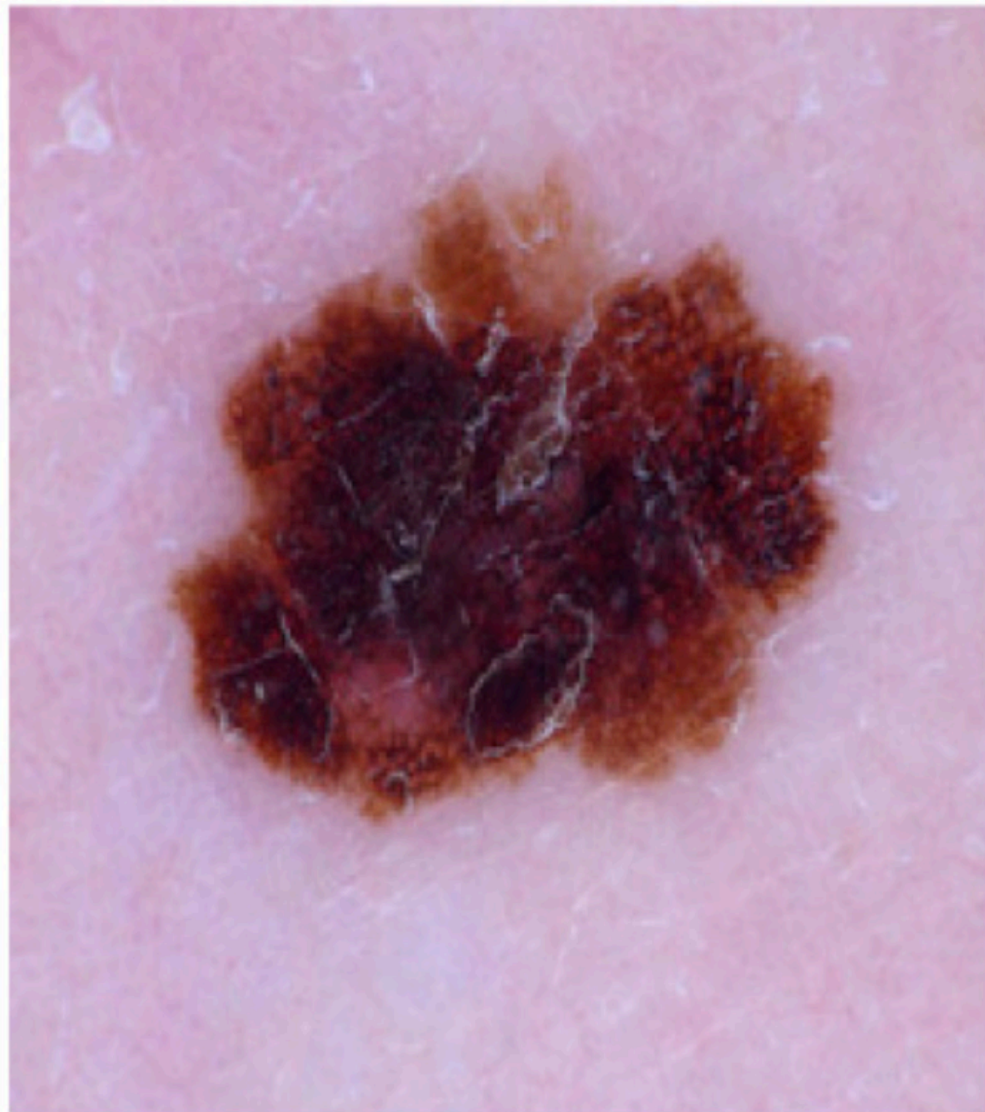
Examples of Supervised Learning

Detecting Skin Cancer from Images

Traditionally, melanoma (skin cancer) has been detected through painful and time-consuming biopsies.

Recently new methods have emerged which use computer-aided detection for early melanoma diagnosis.

The machine learning models were trained using image data.



(a)



(b)

Gouda, W., Sama, N. U., Al-Waakid, G., Humayun, M., & Jhanjhi, N. Z. (2022). Detection of skin cancer based on skin lesion images using deep learning. *Healthcare*, 10(7), 1183. <https://doi.org/10.3390/healthcare10071183>

Examples of Supervised Learning

Detect Numbers in a Handwritten Postal Code

[illegible]

Examples of Supervised Learning

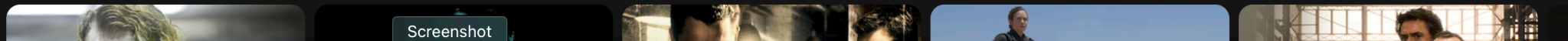
Recommender Systems



Critically-acclaimed Workplace TV Shows



Critically-acclaimed Criminal Investigation Movies



Examples of Supervised Learning

Speech-to-Text, Personal Assistants, Speech Identification



The Basics of Unsupervised Learning

We have a set of features, X , measured on a sample.

We do **not** have an outcome variable, Y .

The objective is more ambiguous

- find groups of samples that behave similarly
- find features that behave similarly
- find linear combinations of features with the most variation

It is difficult to assess the quality of the model

Different from supervised learning, but can be useful as a pre-processing step for supervised learning

Examples of Unsupervised Learning

Image Segmentation

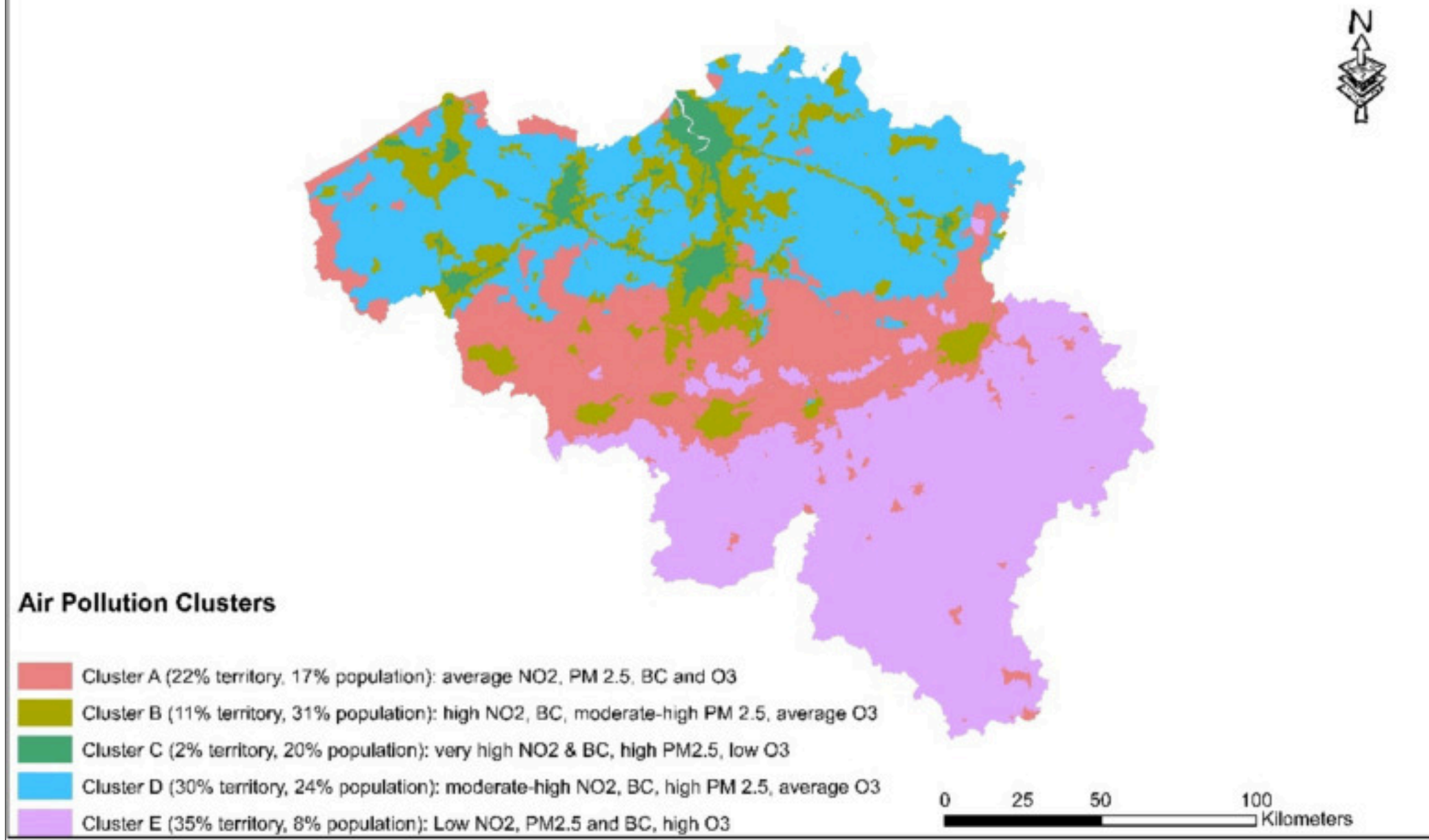
The goal is to cluster pixels into image regions to make it easier to analyze and understand specific parts of the image.



Examples of Unsupervised Learning

Clustering

Grouping unlabeled data points into clusters based on their similarities.



Vandeninden, B., De Clercq, E. M., Devleesschauwer, B., Otavova, M., Bouland, C., & Faes, C. (2024). Cluster pattern analysis of environmental stressors and quantifying

their impact on all-cause mortality in Belgium. *BMC Public Health*, 24(1).

<https://doi.org/10.1186/s12889-024-18011-0>

Examples of Unsupervised Learning

Natural Language Processing

Real world example:

The New York Times

LDA analysis of 1.8M New York Times articles:

music
band
songs
rock
album
jazz
pop
song
singer
night

book
life
novel
story
books
man
stories
love
children
family

art
museum
show
exhibition
artist
artists
paintings
painting
century
works

game
Knicks
nets
points
team
season
play
games
night
coach

show
film
television
movie
series
says
life
man
character
know

theater
play
production
show
stage
street
broadway
director
musical
directed

clinton
bush
campaign
gore
political
republican
dole
presidential
senator
house

stock
market
percent
fund
investors
funds
companies
stocks
investment
trading

restaurant
sauce
menu
food
dishes
street
dining
dinner
chicken
served

budget
tax
governor
county
mayor
billion
taxes
plan
legislature
fiscal

Introduction to Supervised Learning

This Course

In much of this course, we focus on **supervised learning**, where we are given

- **Training Set** $(x_1, y_1), \dots, (x_n, y_n)$ consisting of n samples of
- **features** $x_1, \dots, x_n \in X$ and corresponding
- **outcome** $y_1, \dots, y_n \in Y$

The goal is to **learn a predictor** (function/mapping) $f : X \rightarrow Y$ such that for a new point $x^* \in X$,

$$f(x^*) \approx y^*$$

The Function, f

Imagine the underlying generating mechanism between X and Y as

$$Y = f(X) + \epsilon$$

where ϵ represents some measurement errors and other discrepancies.

We are given the training data D^{train} consisting of n i.i.d samples of (X, Y) following the above model, that is, $(x_1, y_1), \dots, (x_n, y_n)$.

Our goal is to estimate (or learn) the mapping/function f based on D^{train} .

Example: Advertising

We are given information of n products.

For each product, we are given:

- y_i : Sales of product i in 200 different markets
- x_{i1} : TV advertising budget of product i
- x_{i2} : Radio advertising budget of product i
- x_{i3} : Newspaper advertising budget of product i

Suppose the true generating model is

$$\begin{aligned} Y &= f(X) + \epsilon \\ &= \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3 + \epsilon \end{aligned}$$

Knowing f helps to understand how sales changes if we increase one unit of the TV advertising budget (X_1).

Why Estimate f ?

Prediction

Given a new point $X^* = x^*$, $\hat{f}(x^*)$ is the best prediction one can hope for with respect to certain criterion.

Examples:

- Predict a patient's risk of disease based on their characteristics.
- Predict when machinery will break down.

Inference

We want to understand relationships between X and Y .

Examples:

- Which variables are important?
- How does changing X affect Y ?
- What is the strength of association?

Estimating f

Let x_{ij} denote the value of the j^{th} feature for observation i , where $i = 1, 2, \dots, n$ and $j = 1, 2, \dots, p$.

Let y_i denote the response variable for the i^{th} observation.

Then, the training data consists of $D^{train} = \{(x_1, y_1), \dots, (x_n, y_n)\}$, where $x_i = (x_{i1}, \dots, x_{ip})^T$

There are two categories of approaches to estimate f based on D^{train} :

- **Parametric method**
- **Non-parametric method**

Estimating f

Parametric method

Step 1: Make an assumption about the function form, or shape, of f .

Step 2: Use the training data to *fit* (or *train*) the model.

Example: Linear Model

The linear model is an important example of a parametric model.

$Y = f(X) + \epsilon$, with $f(X) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p$.

We would estimate f by

$$\hat{f}_{linear}(X) = \hat{\beta}_0 + \hat{\beta}_1 X_1 + \dots + \hat{\beta}_p X_p.$$

Note: constructing f in this way reduces the problem of estimating a function to that of estimating $(p+1)$ parameters

Estimating f

Parametric method

Example: Linear Model

$$\hat{f}_{linear}(X) = \hat{\beta}_0 + \hat{\beta}_1 X_1 + \dots + \hat{\beta}_p X_p.$$

The procedure of obtaining $\hat{\beta}_0, \dots, \hat{\beta}_p$ is called **fitting**, i.e. fitting the model to the training data.

Even for the same parametric model, there are different ways of fitting! We will come back to this later.

A Toy Example

Linear model

$$\hat{f}_{\text{linear}}(X) = \hat{\beta}_0 + \hat{\beta}_1 X$$

R Code [↺ Start Over](#)

[Run Code](#)

```
1
2  set.seed(314)
3
4  n <- 60
5  x <- runif(n, 1.5, 5.5)
6
7  # True relationship is slightly nonlinear
8  y <- -1.5 + 0.15*x + 0.12*x^2 +
9      rnorm(n, mean = 0, sd = 0.55)
10
11 # Fit a linear model
12 fit_linear <- lm(y ~ x)
13
14 plot(x, y,
15       pch = 1,
16       xlab = "X",
17       ylab = "Y",
18       main = "")
19
```

```
20 abline(fit_linear,  
21        col = "navy",  
22        lwd = 2)
```

A Toy Example

Quadratic model

$$\hat{f}_{\text{quad}}(X) = \hat{\beta}_0 + \hat{\beta}_1 X + \hat{\beta}_2 X^2$$

R Code [↺ Start Over](#)

[Run Code](#)

```
1
2 # Fit a quadratic model
3 fit_quad <- lm(y ~ x + I(x^2))
4
5 x_grid <- seq(min(x), max(x), length.out = 200)
6
7 y_hat <- predict(
8   fit_quad,
9   newdata = data.frame(x = x_grid)
10 )
11
12 plot(x, y,
13       pch = 1,
14       xlab = "X",
15       ylab = "Y",
16       main = "")
17
18 lines(x_grid, y_hat,
19       col = "maroon",
```

20

lwd = 2)

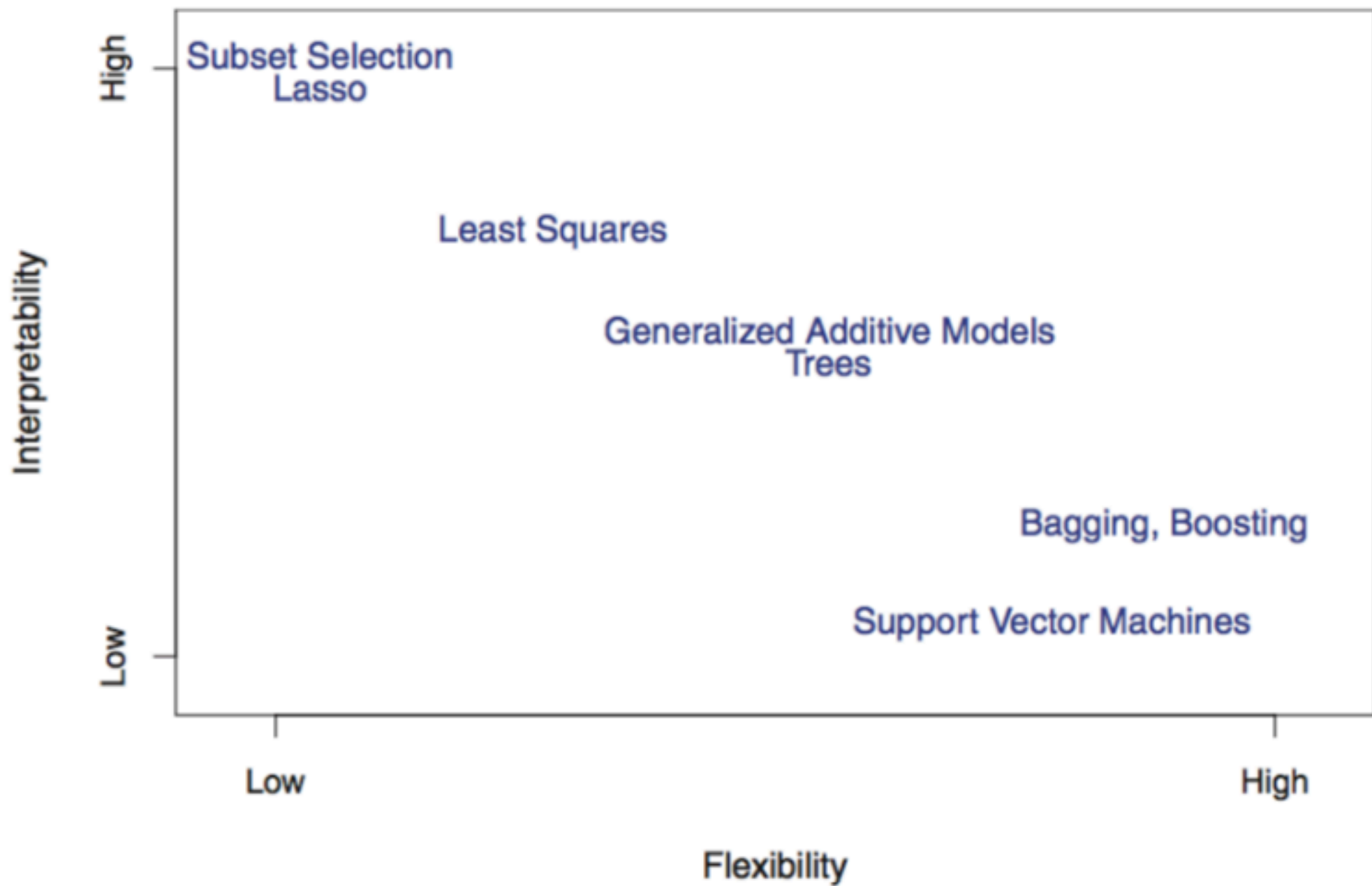
Trade off Between Complexity and Interpretability

The more complex the model, the more flexible to fit f , but less interpretable.

Linear models are easy to interpret, neural networks are not.

Interpretation means to understand how the predictors X contribute to predicting Y .

Trade off Between Complexity and Interpretability



Estimating f

Non-Parametric method

These methods do not make explicit assumptions about the functional form of f . Instead, we let the data determine the shape more directly.

Example: Nearest Neighbours

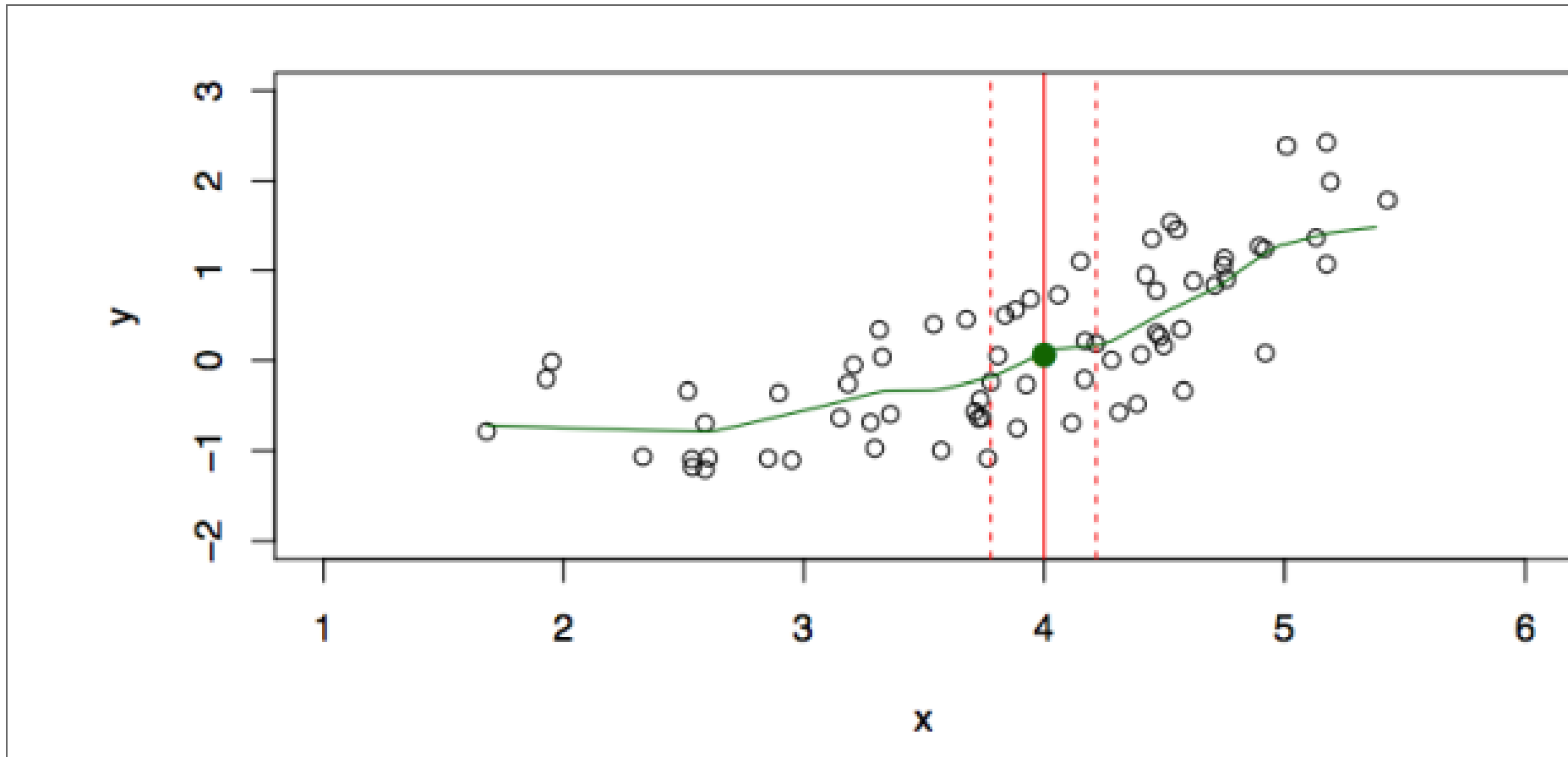
The goal is to predict $X = x$ using $N(x)$, which is some neighbourhood of x .

$$\hat{f}(x) = \frac{1}{|N(x)|} \sum_{i: x_i \in N(x)} y_i$$

In simple terms, the prediction is the average response of nearby observations.

Estimating f

Non-Parametric method



Estimating f

Non-Parametric method

There are other, more sophisticated methods as well, such as kernel regression and spline regression.

Pros

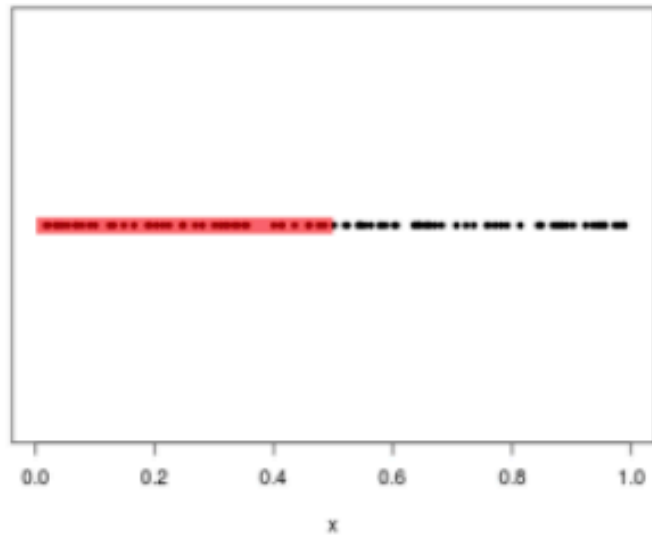
- we don't make assumptions on f
- it's good at predicting for large n and small p , e.g. $p \leq 4$

Cons

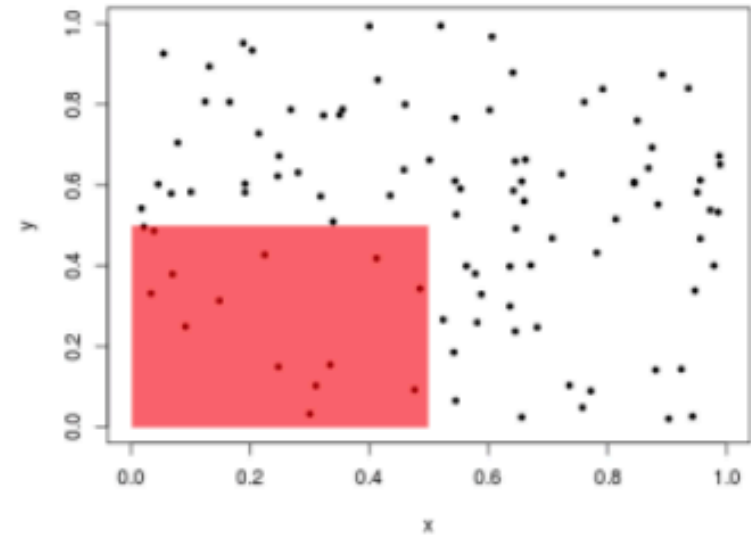
- poor performance when p is large
- **curse of dimensionality**: There are very few data points in the nearby neighbours when p is large

Curse of Dimensionality

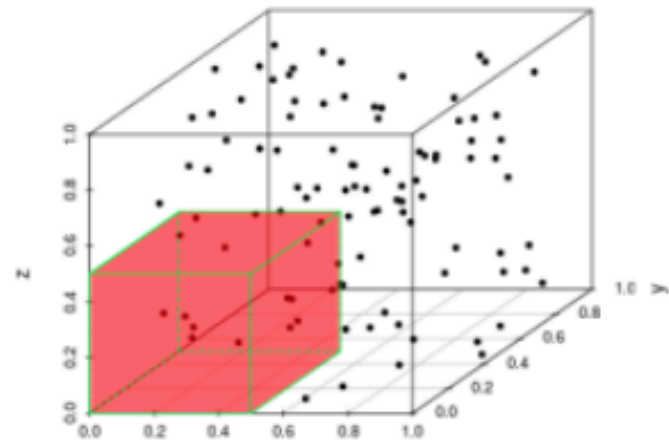
1-D: 42% of data captured.



2-D: 14% of data captured.

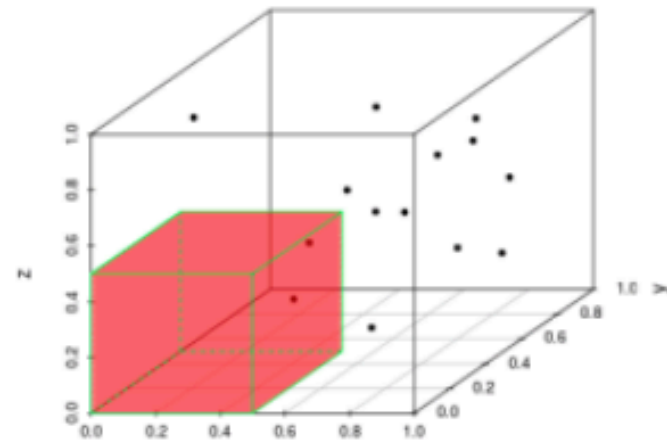


3-D: 7% of data captured.



4-D: 3% of data captured.

$t = 0$



Model Selection

Model Selection

“All models are wrong, but some are useful” George Box

We need a systematic way of choosing the best model.

We start with the **regression** problems where Y is continuous.

Model Selection: Regression

Recall the setup:

$$Y = f(X) + \epsilon$$

and D^{train} contains n i.i.d samples of (X, Y) .

Given any $\hat{f}$, ideally, we want to evaluate $\hat{f}$ by the expected **mean squared error (MSE)**

$$E \left[(Y^* - \hat{f}(X^*))^2 \right]$$

where

- (X^*, Y^*) is a new random pair that is independent of D^{train}
- the expectation is taken w.r.t the random pair (X^*, Y^*) , as well as the randomness in $\hat{f}$.

Model Selection: Regression

We cannot compute the expected MSE as we do not know either the distribution of (X^*, Y^*) , or that of D^{train} .

One natural option is to use D^{train} to approximate the expectation by

$$MSE(\hat{f}) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{f}(x_i))^2$$

This is called the **training MSE**, as it uses D^{train}

However, it is **NOT** a valid metric of the fit for $\hat{f}$ since it always favours more complex models.

Model Selection: Regression

Test data refers to the data which is not used to train the statistical model

Suppose we have the test data D_{test} containing $(x_{T1}, y_{T1}), \dots, (x_{Tm}, y_{Tm})$. Then, we can compute the **test MSE** as

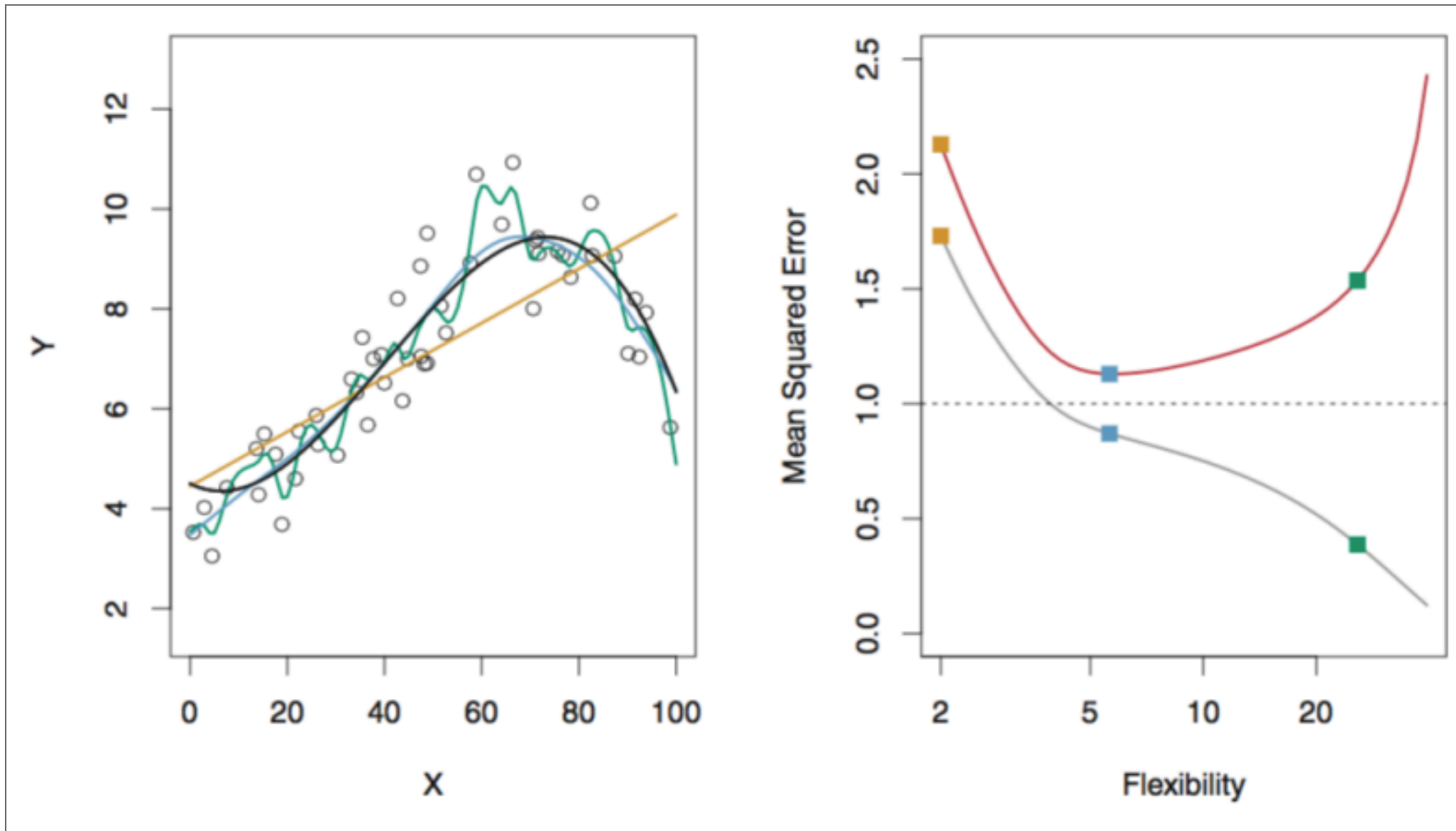
$$MSE_T(\hat{f}) = \frac{1}{m} \sum_{i=1}^m (y_{Ti} - \hat{f}(x_{Ti}))^2$$

Instead of using the training MSE, we should look at the test MSE. I.e. we select the model which yields the smallest test MSE.

How to calculate $MSE_T(\hat{f})$?

- If test data is available, we can directly compute it
- Otherwise, we use a resampling technique called *cross-validation* which we will discuss in another lecture

Training MSE vs. Test MSE



Left Plot: Data simulated from f , shown in black. Three estimates of f are shown: the linear regression line (orange), and two non parametric fits (blue and green).

Right Plot: Training MSE (grey), test MSE (red), and minimum test MSE over all possible methods coming from ϵ (dashed line)

Overfitting

The training MSE decreases as $\hat{f}$ gets more complex.

A highly complex $\hat{f}$ can lead to a phenomenon known as **overfitting** the data, which essentially means it follows the noise ϵ too closely.

A Simple Example: $\hat{f}(x_i) = y_i$ for all $1 \leq i \leq n$

Overfitting Example

R Code

↺ Start Over

Run Code

```
1  set.seed(314)
2
3  n <- 30
4
5  x <- sort(runif(n, 0, 10))
6
7  # True relationship
8  f <- function(x) 2 + 0.5*x
9
10 # Observed response = signal + noise
11 y <- f(x) + rnorm(n, mean = 0, sd = 1.5)
12
13 plot(x, y,
14       pch = 19,
15       xlab = "X",
16       ylab = "Y")
17
18 # True f(x)
19 curve(f(x),
20       add = TRUE,
21       lwd = 3,
22       col = "black")
```

The points do not lie exactly on f because of the random noise, ϵ .

Overfitting Example

R Code

↻ Start Over

Run Code

```
1
2 # Fit a simple linear model
3 fit_simple <- lm(y ~ x)
4
5 # Fit a much more flexible model
6 fit_flexible <- lm(y ~ poly(x, 12))
7
8 x_grid <- seq(min(x), max(x), length.out = 500)
9
10 plot(x, y,
11       pch = 19,
12       xlab = "X",
13       ylab = "Y")
14
15 # True relationship
16 lines(x_grid,
17       f(x_grid),
18       lwd = 3,
19       col = "black")
20
21 # Simple model
22 lines(x_grid,
23       predict(fit_simple,
24             newdata = data.frame(x = x_grid)),
```

```
25     lwd = 2,  
26     col = "navy")  
27  
28 # Flexible model  
29 lines(x_grid,  
30       predict(fit_flexible,  
31               newdata = data.frame(x = x_grid)),  
32       lwd = 2,  
33       col = "maroon")  
34  
35 legend("topleft",  
36       legend = c("True f(x)",  
37                 "Linear fit",  
38                 "Flexible fit"),  
39       col = c("black", "navy", "maroon"),  
40       lwd = c(3, 2, 2),  
41       bty = "n")
```

Back to the Test MSE

There is a tradeoff between the test MSE and the flexibility (complexity) of the fitted model, $\hat{f}$.

Is there a universal rule or explanation about this?

Bias-Variance Decomposition

Let (X^*, Y^*) be a new random pair (independent from D^{train}) following $Y^* = f(X^*) + \epsilon^*$ with $E[\epsilon^*] = 0$.

For any estimator $\hat{f}$ (obtained from D^{train}), its conditional expected MSE at any $X^* = x^*$ is

$$\begin{aligned} \mathbb{E} \left[\left(Y_* - \hat{f}(X_*) \right)^2 \mid X_* = x_* \right] &= \underbrace{\text{Var} \left(\hat{f}(x_*) \right)}_{\text{Variance}} + \underbrace{\left(\mathbb{E} \left[\hat{f}(x_*) \right] - f(x_*) \right)^2}_{\text{Bias}^2} + \underbrace{\text{Var}(\epsilon_*)}_{\text{Irreducible}} \\ &\geq \text{Var}(\epsilon_*). \end{aligned}$$

- the first expectation is over ϵ^* as well as D^{train}
- the expected MSE $\geq$ the irreducible error
- an ideal $\hat{f}$ should minimize the expected MSE

Bias-Variance Tradeoff

Variance: how much $\hat{f}$ would change if we estimated it using a different training data set

Bias: the systematic difference between our estimated relationship $\hat{f}$ and the true relationship f

- **Example:** If the true relationship is nonlinear but we fit a linear model, a straight line cannot fully capture the true relationship, resulting in bias.

Bias-Variance Tradeoff

As the complexity (i.e. flexibility) of $\hat{f}$ increases, the variance of $\hat{f}$ typically increases whereas its bias decreases

- The variance of any fitted model also depends on the sample size, n , and is roughly proportional to $\frac{\text{complexity of } \hat{f}}{n}$
- When n is small, a fitted model with high complexity performs poorly due to large variance
- When n is large enough, a more complex fitted model tends to perform better as they have smaller bias than simpler models

Bias-Variance Tradeoff

Choosing the complexity of $\hat{f}$ based on the expected MSE involves a bias-variance tradeoff.

When two models, $\hat{f}_1$ and $\hat{f}_2$ have similar expected MSEs, we usually prefer the *more parsimonious* (less complex) one.

Why? The additional complexity provides little improvement in prediction, while potentially making the model less interpretable and more variable.

Model Selection: Classification

The MSE is only appropriate for a **quantitative outcome**.

One metric we can use for categorical or ordinal outcomes is the **expected error rate**, defined as

$$E[1\{Y \neq \hat{f}(X)\}].^1$$

¹($\{\}$) is the indicator function. ($\{A\}=1$) if (A) is true and ($\{A\}=0$) otherwise.

Model Selection: Classification

Analogously, the **training error rate** is

$$\frac{1}{n} \sum_{i=1}^n 1\{y_i \neq \hat{f}(x_i)\}$$

And the **test error rate** is

$$\frac{1}{m} \sum_{i=1}^m 1\{y_{Ti} \neq \hat{f}(x_{Ti})\}$$

Of course, there also exist other metrics that can be used for classification problems.

Summary

- Supervised vs. unsupervised learning
- The goal with supervised learning is to estimate a function f based on the training data for prediction or inference
- To estimate f , we can use parametric (assume a model) or non-parametric (data-driven) methods
- To choose the best model, we use a metric for evaluating a given $\hat{f}$
- The best model yields the smallest expected test MSE (or error rate for classification problems)
- **Bias-Variance tradeoff:** a more complex/flexible $\hat{f}$ has a smaller bias but larger variance

Next Class: Some algorithms of computing $\hat{f}$!

Announcements: No tutorial on Monday, Sept 14.

